

Performance Comparison of Scheduling Policies in XV6

Sushant Kumar

October 14, 2024

1 Introduction

This report discusses the implementation of the Lottery-Based Scheduling (LBS) and Multi-Level Feedback Queue (MLFQ) scheduling policies in the XV6 operating system. The default scheduling policy in XV6 is round-robin scheduling. The modifications made to the XV6 kernel include adding two new scheduling policies that aim to enhance process management and CPU utilization.

2 Implementation Overview

2.1 Lottery-Based Scheduling

In the LBS implementation, each process is assigned a number of tickets, determining its probability of being scheduled. The scheduler randomly selects a process based on its ticket count. Additionally, I modified the 'proc' structure to include ticket information and adjusted the scheduling algorithm to prioritize processes based on their tickets. Moreover, if there is a tie in the number of tickets that 2 processes have, their arrival time has been taken into account (in the main for loop).

2.2 Multi-Level Feedback Queue

The MLFQ scheduler consists of four priority queues, with each queue having a different time slice. Processes move between queues based on their CPU usage. The priority boosting mechanism was also implemented to prevent starvation, ensuring that processes return to the highest priority queue after a certain number of ticks.

1. On the initiation of a process, push it to the end of the highest priority queue (priority 0):

Priority 0 initialization has be done in allocproc() when processes become RUNNABLE.

2. Always run the processes that are in the highest priority queue

that is not empty:

This is implemented in the queue selection logic:

```

for (int prio = 0; prio < 4; prio++) // Iterate through priority levels
for (p = proc; p < & proc[NPROC]; p++)
acquire(p->lock);
if (p->state == RUNNABLE & & p->priority == prio)
selected_proc = p; // Found the process to run
break; // Stop searching once we find the first runnable process

release(& p->lock);

if (selected_proc)
break; // Break outer loop if a process has been selected

```

This logic goes through the queues in order from priority 0 to 3 and selects the first runnable process from the highest-priority non-empty queue.

3. Preempt the current running process if a higher priority process becomes available:

In my code, this feature is indirectly implemented. If a higher-priority process becomes runnable, the scheduler will select that process in the next tick since it always checks the highest-priority queue first (mid-tick preemption hasn't been handled).

4. When the process completes, it leaves the system:

This gets implicitly handled as exit sets the process state to ZOMBIE and only RUNNABLE processes participate in my scheduler logic.

5. Preempt and move a process to the next lower priority queue if it uses its entire time slice:

This is implemented in my time slice exhaustion logic:

```

if (selected_proc->time_slices == 0)
if (selected_proc->priority < 3)
selected_proc->priority++; // Demote to the next lower priority

// Set a new time slice for the demoted priority level
switch (selected_proc->priority)
case 1:
selected_proc->time_slices = 4;
break;
case 2:
selected_proc->time_slices = 8;
break;
case 3:
selected_proc->time_slices = 16;

```

break;

6. If a process voluntarily relinquishes CPU (e.g., for I/O), it leaves the queue, and when the process becomes ready to consume CPU then priority remains the same:

Implicitly handled by sleep() which changes state from RUNNABLE to SLEEPING and wakeup() which reverses this change in state without changing the priority.

Also if a process voluntarily yields then the yield function has been modified to reset ticks for the process in that level.

7. A round-robin scheduler should be used for processes at the lowest priority queue:

I handle round-robin for the lowest-priority queue indirectly by placing processes back into the same queue after they use their time slices:

```
if (selectedproc > priority == 3)
    selectedproc > timeslices = 16; // Stay in the same queue with 16 ticks
```

8. To prevent starvation, implement priority boosting:

```
if (tickssincelastboost >= boostticks)
    for (p = proc; p < & proc[NPROC]; p++)
        acquire(& p->lock);
    if (p->state == RUNNABLE) p->priority = 0; // Moves all processes back to the highest queue
    release(& p->lock);
```

```
tickssincelastboost = 0; // Reset the boost timer
```

3 Performance Comparison

To evaluate the performance of the implemented scheduling policies, I used the ‘schedulertest’ command which simulates 3 I/O jobs and 3 CPU bound jobs to measure the average waiting and running times for processes on a single CPU. The results are summarized in Table 1.

4 Implications of Arrival Time in Lottery-Based Scheduling

Adding the arrival time in the lottery-based scheduling policy allows for better process management, as it enables the scheduler to consider the order of

Table 1: Performance Comparison

Policy	Average Running Time (ticks)	Average Waiting Time (ticks)
Default (Round-Robin 1 CPU)	10	85
Lottery-Based Scheduling (3 CPUs)	9	85
Lottery-Based Scheduling (1 CPU)	11	84
Multi-Level Feedback Queue (1 CPU)	6	89

process arrival in addition to ticket allocation. This can help reduce starvation for processes that may otherwise have less chance of being scheduled. However, pitfalls may include increased complexity in managing process states and potentially longer waiting times for processes with fewer tickets if not properly balanced. If all processes have the same number of tickets, the selection becomes random, leading to round-robin-like behavior among those processes, potentially negating the advantages of the lottery-based approach.

5 MLFQ Analysis

6 processes with 3 being I/O bound and 3 being CPU bound were spawned by `schedulertest` command and their priority was analyzed until termination.

6 Conclusion

In conclusion, the implementation of LBS and MLFQ in XV6 provides valuable insights into process scheduling efficiency. The comparison of performance metrics highlights the advantages of these policies over the default round-robin approach.

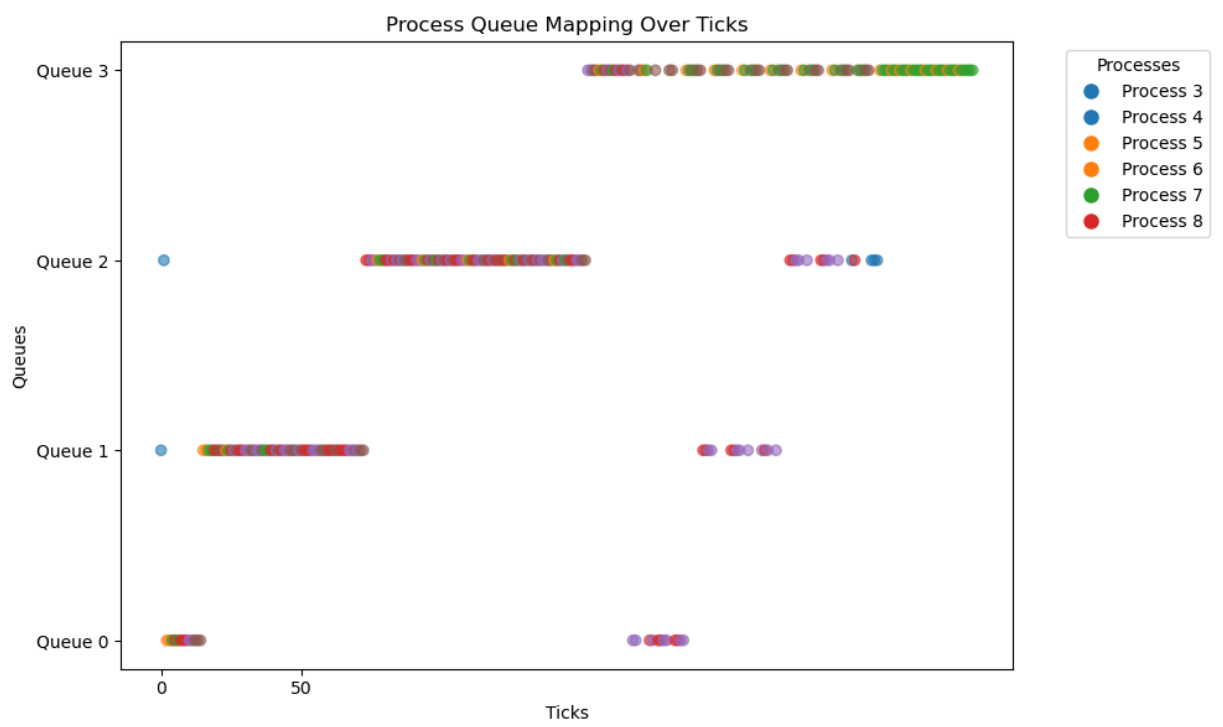


Figure 1: Scatter Plot

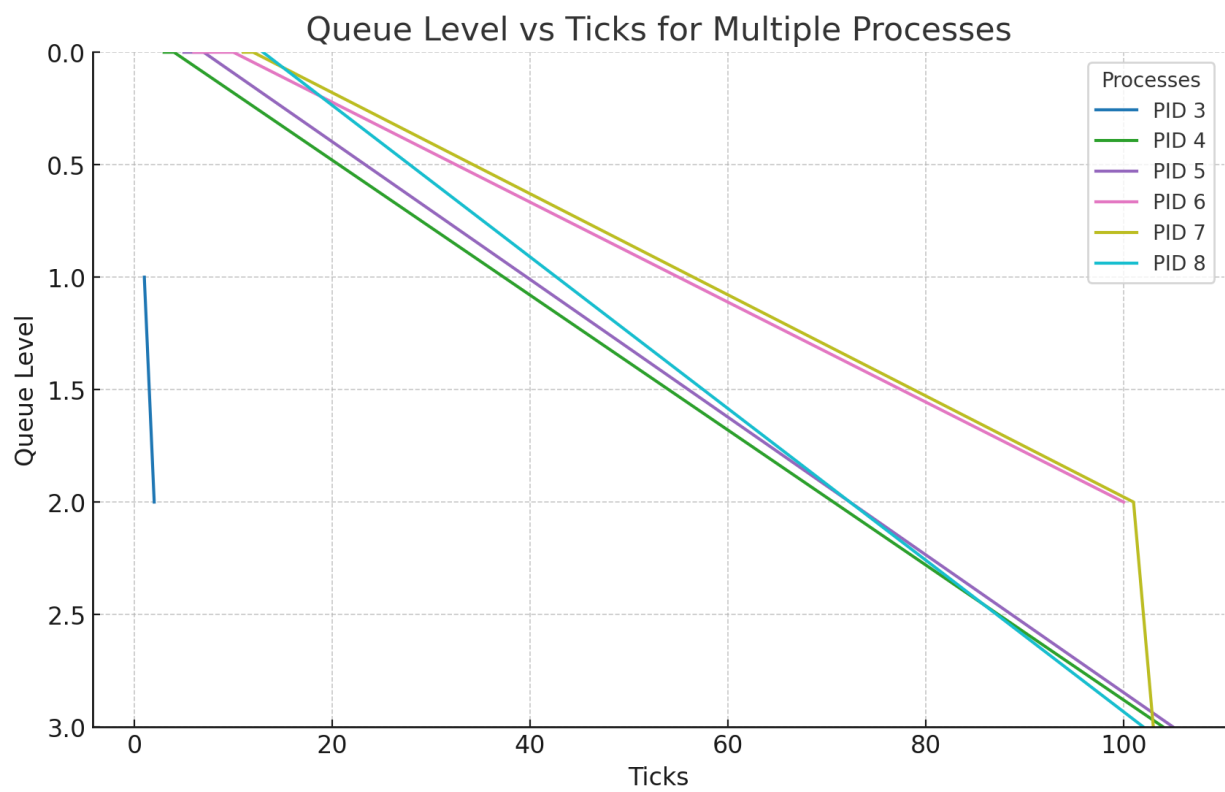


Figure 2: Line Plot (averaged and aggregated)

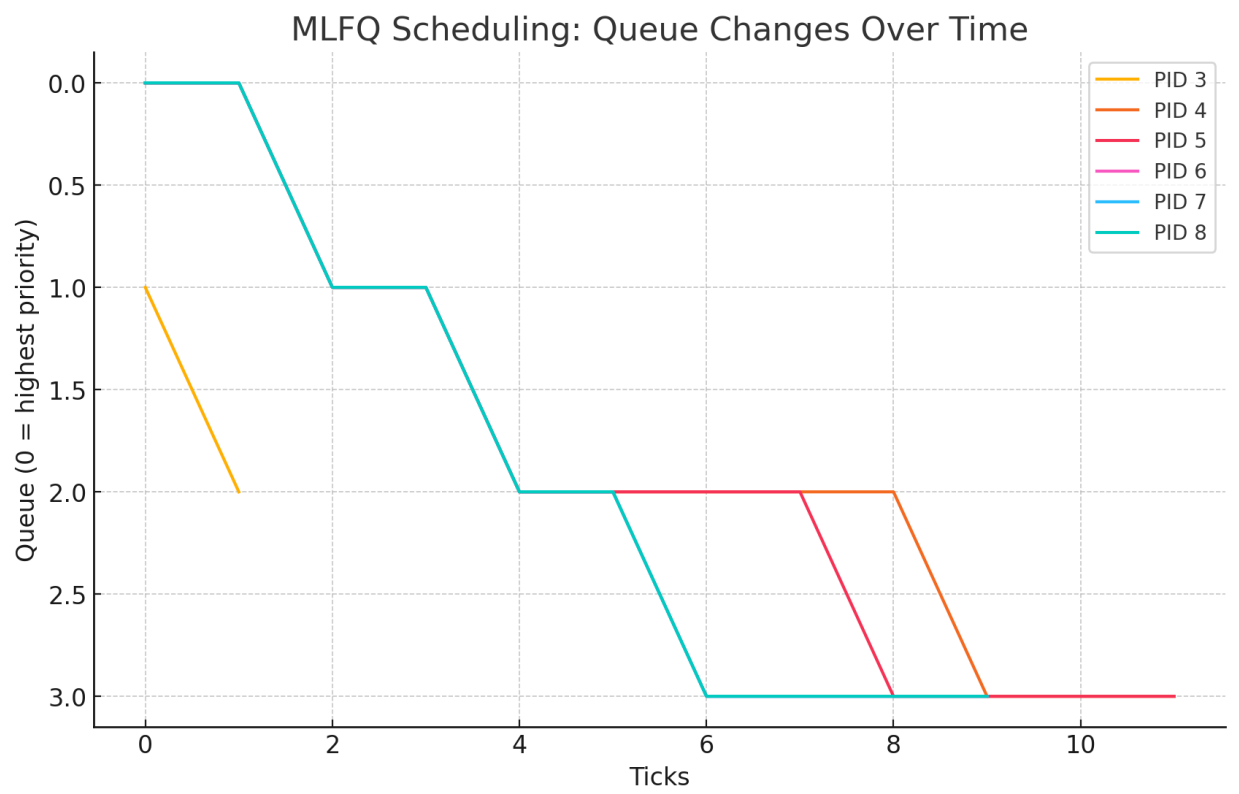


Figure 3: Magnified line plot for processes (only for visualization)